Real-time Bitcoin Price Analysis Using Kubeflow

Project Goal:

To fetch real-time Bitcoin prices at regular intervals, store them in a database, and prepare for future time series analysis using Kubeflow Pipelines.

Technologies Used and Work Done:

1. Minikube Setup

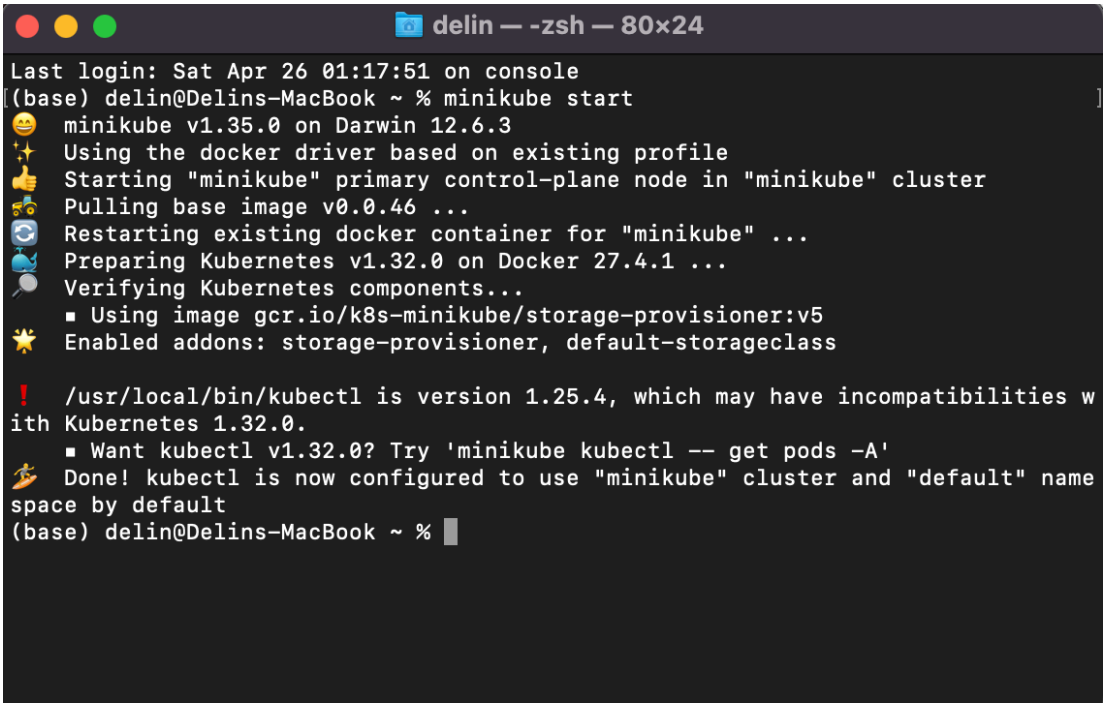
Description: Started a local Kubernetes cluster using Minikube to host Kubeflow and other services.

Command used:

```
minikube start
```

Result:

- Minikube cluster started successfully.



2. Kubeflow Installation and Verification

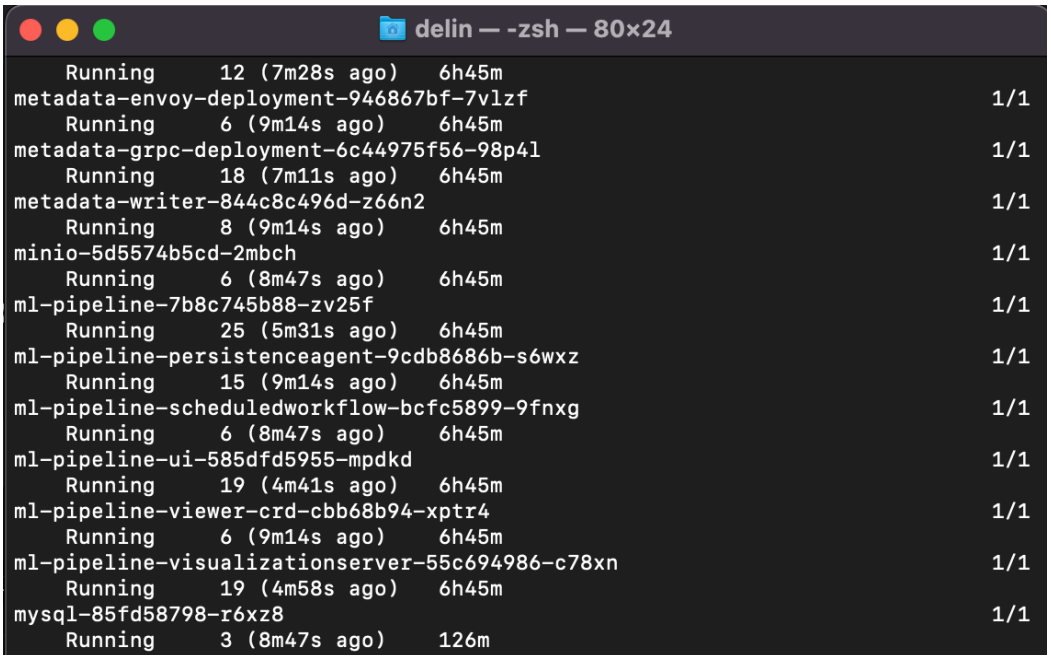
Description: Verified that Kubeflow services and Pods were running.

Command used:

```
kubectl get pods -n kubeflow
```

Result:

- Kubeflow core components like ml-pipeline-ui, cache-server, workflow-controller, mysql, etc. were running.



3. Docker Setup

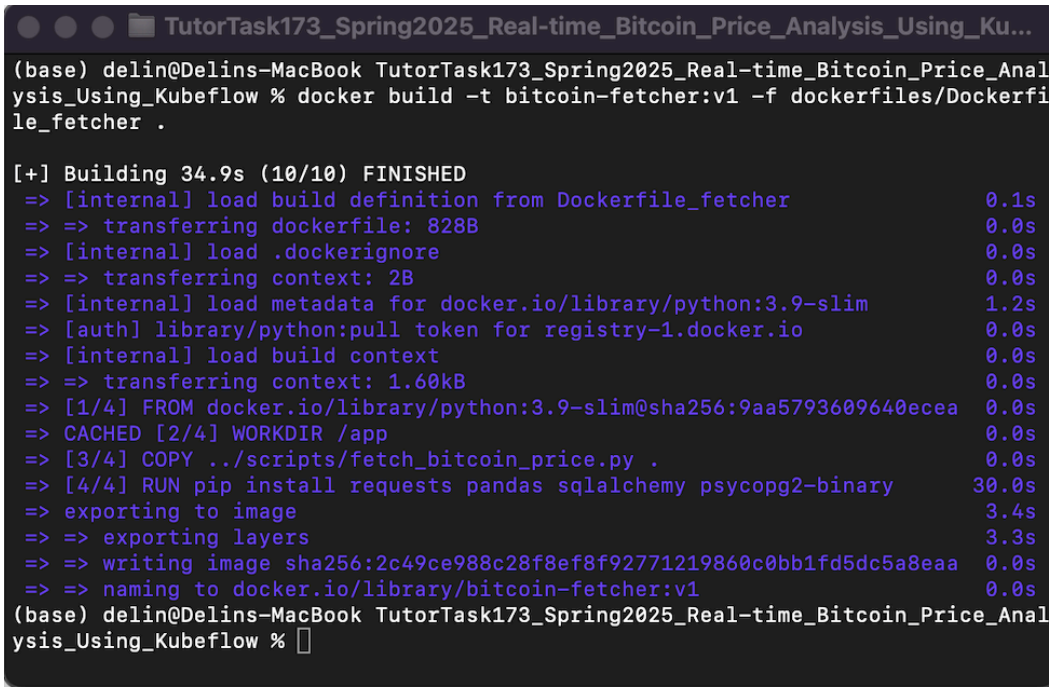
Description: Built a custom Docker image bitcoin-fetcher:v1 containing a script to fetch Bitcoin prices from the API and write to TimescaleDB.

Command used:

```
docker build -t bitcoin-fetcher:v1 -f dockerfiles/
Dockerfile_fetcher
```

Result:

- Successfully created and stored the docker image locally inside Minikube.



Dockerfile_fetcher Used:

```
# Use a small Python image
FROM python:3.9-slim

# Set working directory inside the container
WORKDIR /app

# Copy the Bitcoin price fetcher script into the container
COPY ../scripts/fetch_bitcoin_price.py .

# Install required Python libraries
RUN pip install requests pandas sqlalchemy psycopg2-binary

# Set environment variables default values (optional safety)
ENV DB_USER=postgres
ENV DB_PASSWORD=yourpassword
ENV DB_HOST=localhost
ENV DB_PORT=5432
ENV DB_NAME=bitcoin_db
ENV COINGECKO_API_KEY=your_coingecko_api_key

# Command to run the script
CMD ["python", "fetch_bitcoin_price.py"]
```

4. Python Script for Fetching Bitcoin Price

File: fetch_bitcoin_price.py

```
import os
import requests
import pandas as pd
from sqlalchemy import create_engine
from datetime import datetime

# Fetch Bitcoin price
def fetch_bitcoin_price():
    url = "https://api.coingecko.com/api/v3/simple/price"
    params = {
        "ids": "bitcoin",
        "vs_currencies": "usd"
    }
    headers = {
        "x-cg-pro-api-key": os.getenv("COINGECKO_API_KEY")
    }
    response = requests.get(url, params=params,
headers=headers)
    data = response.json()
    price = data["bitcoin"]["usd"]
    return price

# Save to TimescaleDB
def save_to_database(price):
    db_user = os.getenv("DB_USER")
    db_password = os.getenv("DB_PASSWORD")
    db_host = os.getenv("DB_HOST")
    db_port = os.getenv("DB_PORT")
    db_name = os.getenv("DB_NAME")

    connection_string = f"postgresql://{db_user}:
{db_password}@{db_host}:{db_port}/{db_name}"
    engine = create_engine(connection_string)

    df = pd.DataFrame({
        "timestamp": [datetime.utcnow()],
        "price": [price]
    })

    with engine.connect() as conn:
```

```
        df.to_sql("bitcoin_prices", conn, if_exists="append",
index=False)

# Main execution
if __name__ == "__main__":
    price = fetch_bitcoin_price()
    save_to_database(price)
    print(f"[Saved] Bitcoin price ${price} at
{datetime.utcnow()}")
```

5. Pipeline Upload

Description: Created a Kubeflow Pipeline YAML file and uploaded it through Kubeflow UI.

Steps performed:

- Opened Kubeflow Dashboard (ml-pipeline-ui).
- Uploaded compiled `bitcoin_pipeline.yaml`.

Python Script for Creating Pipeline YAML

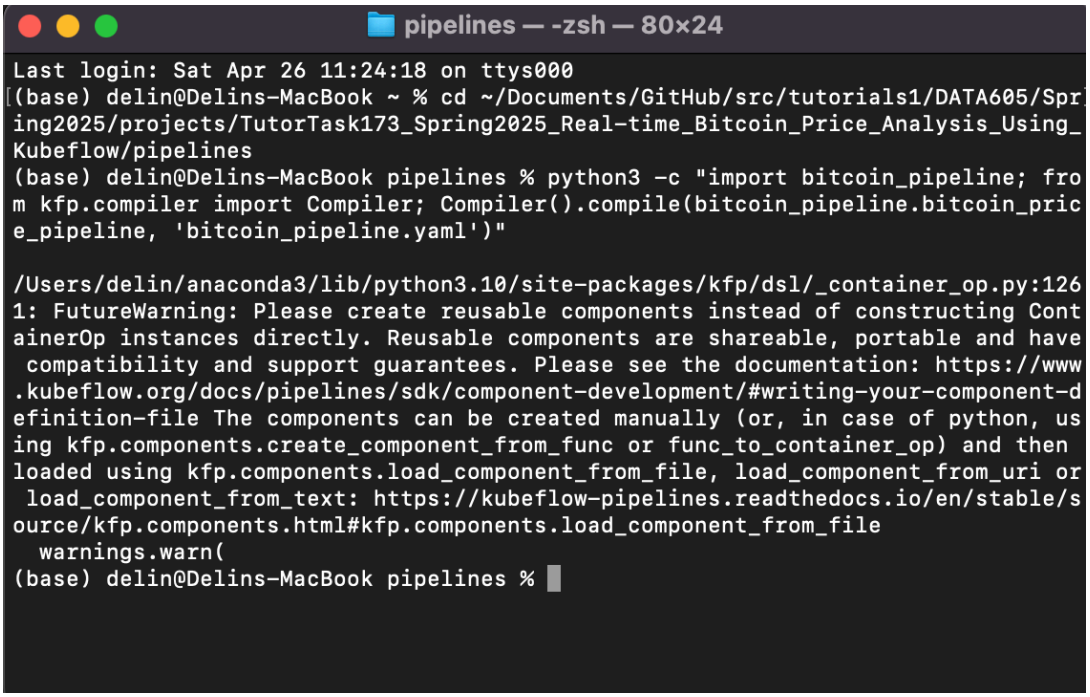
File: bitcoin_pipeline.py

```
# bitcoin_pipeline.py

from kfp import dsl
from kubernetes import client as k8s_client

def bitcoin_price_pipeline():
    fetch_op = dsl.ContainerOp(
        name="Fetch Bitcoin Price",
        image="bitcoin-fetcher:v1",
        command=["python", "fetch_bitcoin_price.py"]
    )


fetch_op.container.add_env_variable(k8s_client.V1EnvVar(name=
'DB_USER', value='postgres'))

fetch_op.container.add_env_variable(k8s_client.V1EnvVar(name=
'DB_PASSWORD', value='testpass'))
```

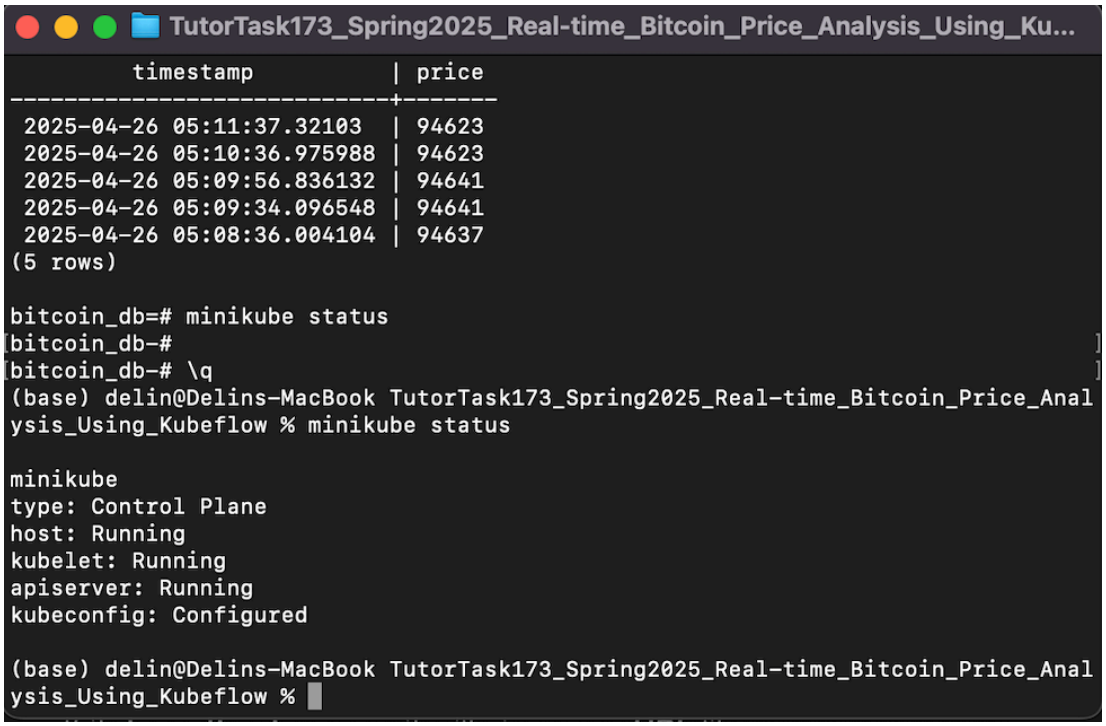
```
fetch_op.container.add_env_variable(k8s_client.V1EnvVar(name=
'DB_HOST', value='host.docker.internal'))

fetch_op.container.add_env_variable(k8s_client.V1EnvVar(name=
'DB_PORT', value='5432'))

fetch_op.container.add_env_variable(k8s_client.V1EnvVar(name=
'DB_NAME', value='bitcoin_db'))

fetch_op.container.add_env_variable(k8s_client.V1EnvVar(name=
'COINGECKO_API_KEY', value='CG-KufNKPvFrCUCFUWHxW6yyXTM'))
```

Command used to compile:

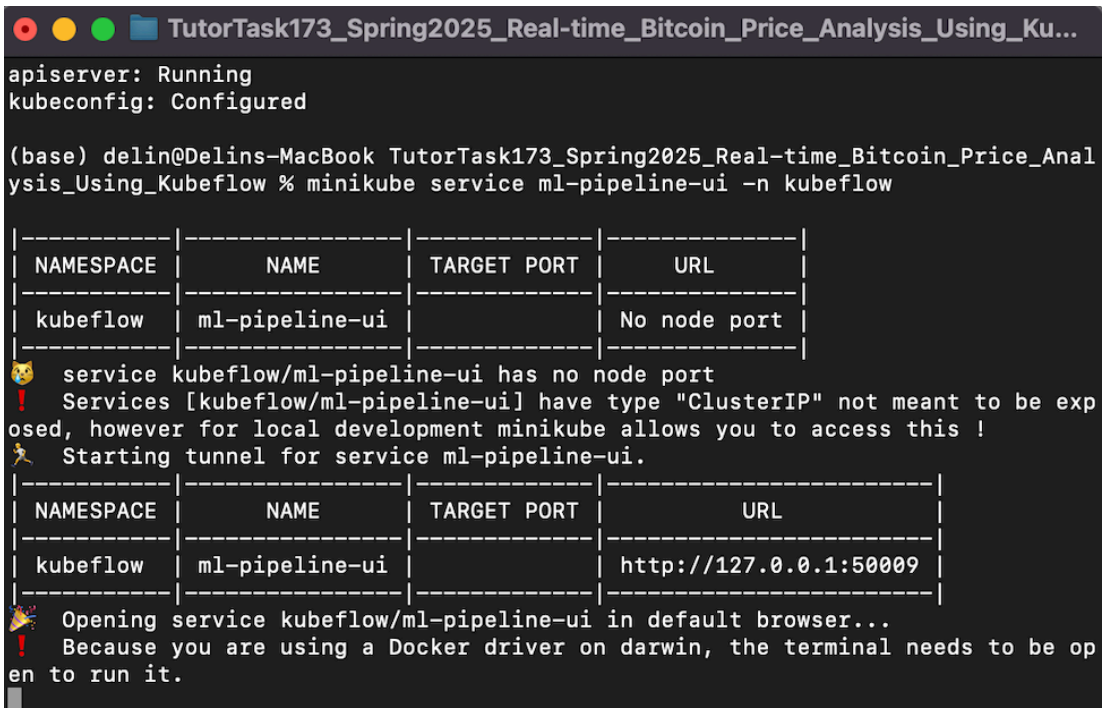
```
python3 -c "import bitcoin_pipeline; from kfp.compiler import
Compiler;
Compiler().compile(bitcoin_pipeline.bitcoin_price_pipeline,
‘bitcoin_pipeline.yaml')"
```



Make sure Minikube is running



Open the Kubeflow Pipelines Dashboard in your browser



6. Create Recurring Run

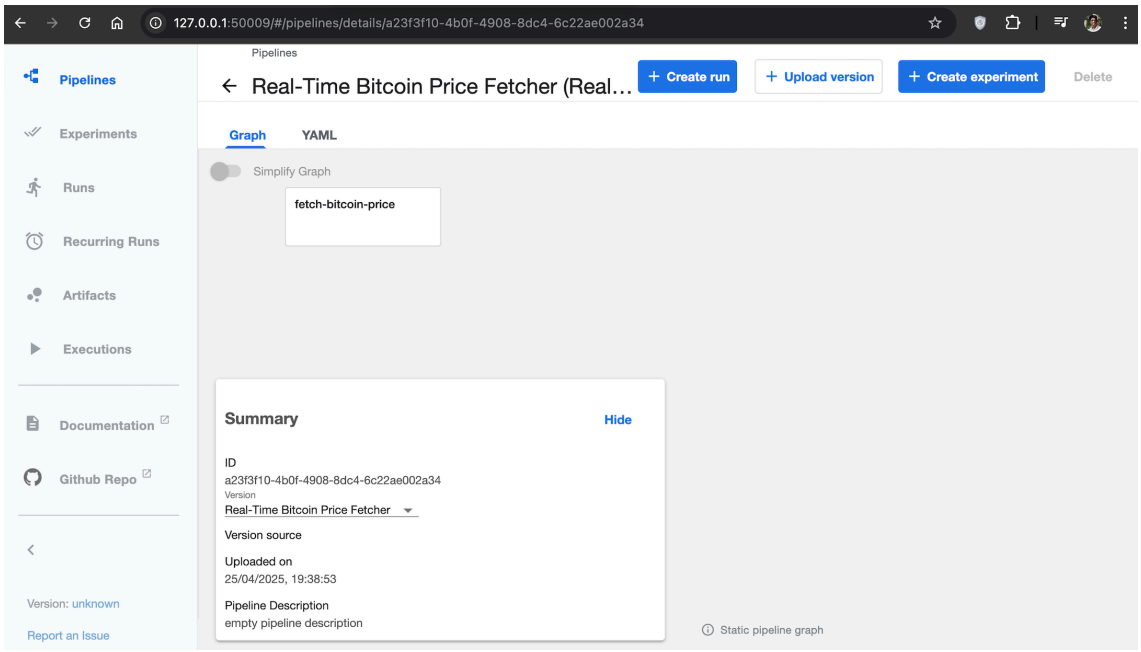
Description: Created a recurring run that triggers every 1 minute.

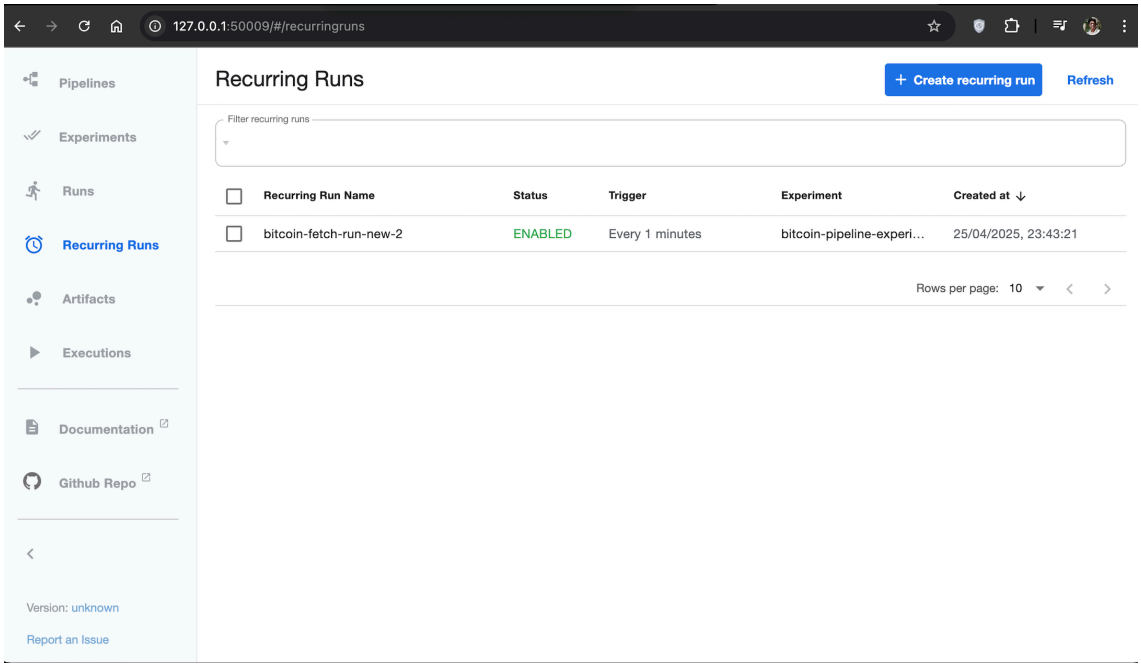
Steps:

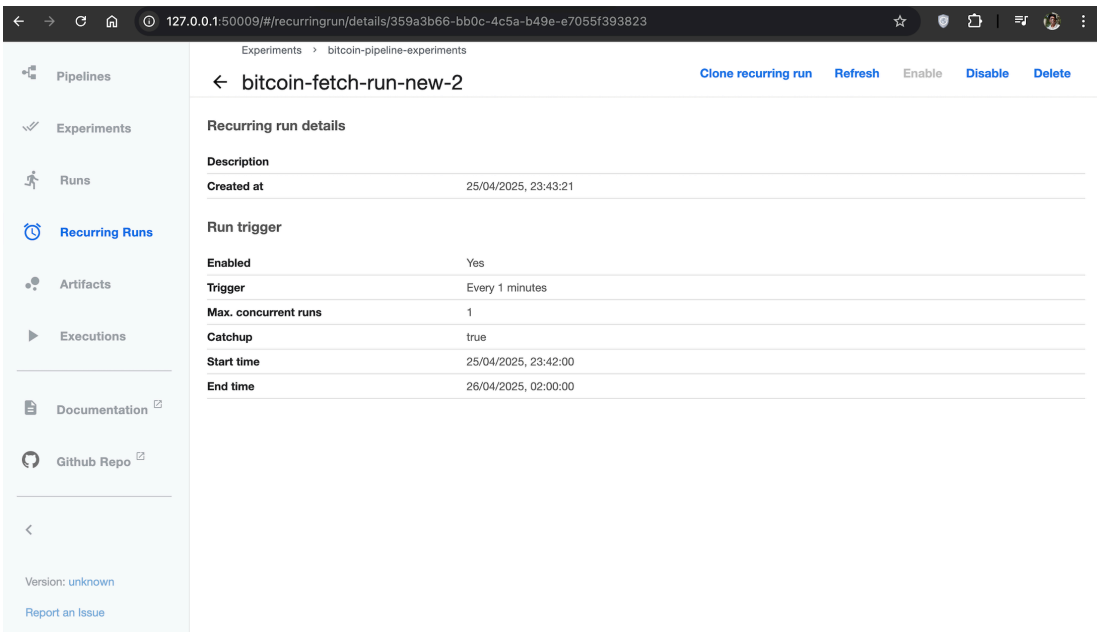
- Opened Kubeflow Pipelines dashboard.
- Created a new Recurring Run.
- Set the pipeline trigger interval to 1 minute.

Result:

- Bitcoin price is now fetched and stored every minute into TimescaleDB.







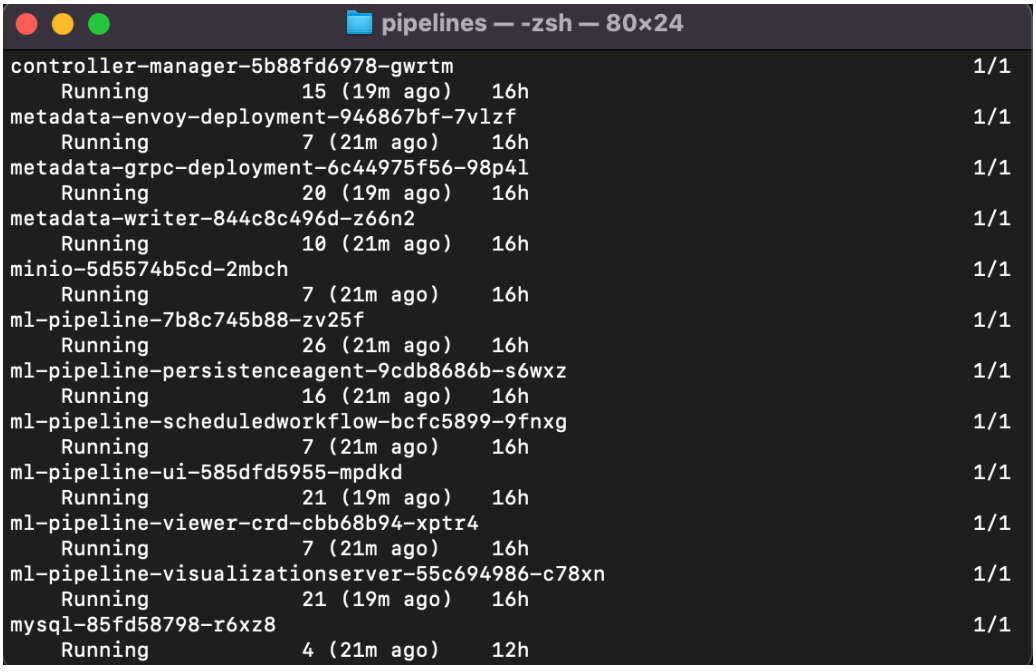
7. Database Setup

Description:

- Deployed MySQL inside Kubeflow.
- Initially used TimescaleDB inside Docker to validate local fetching.
- Later relied on Minikube-internal MySQL Pod (mysql-85fd58798-r6xz8) for database.

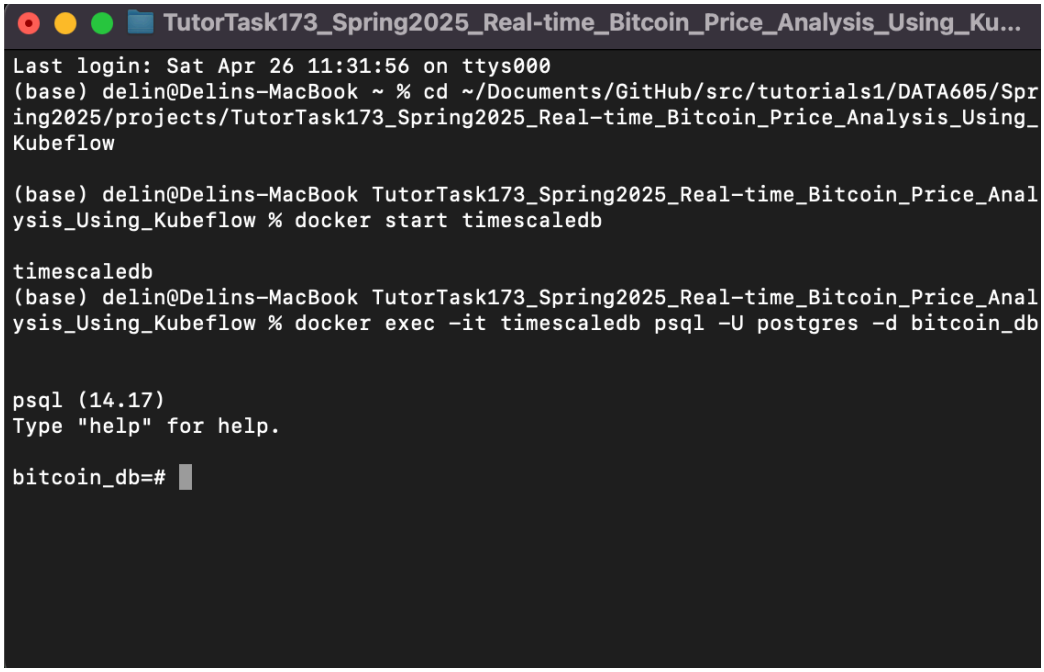
Command to check Pods:

```
kubectl get pods -n kubeflow
```



Database Connection Commands: Docker TimescaleDB:

```
docker exec -it timescaledb psql -U postgres -d bitcoin_db
```



File: mysql-pv.yaml

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: mysql-pv
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  hostPath:
    path: "/data/mysql"
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mysql-pv-claim
  namespace: kubeflow
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
storage: 1Gi
```

8. Data Validation

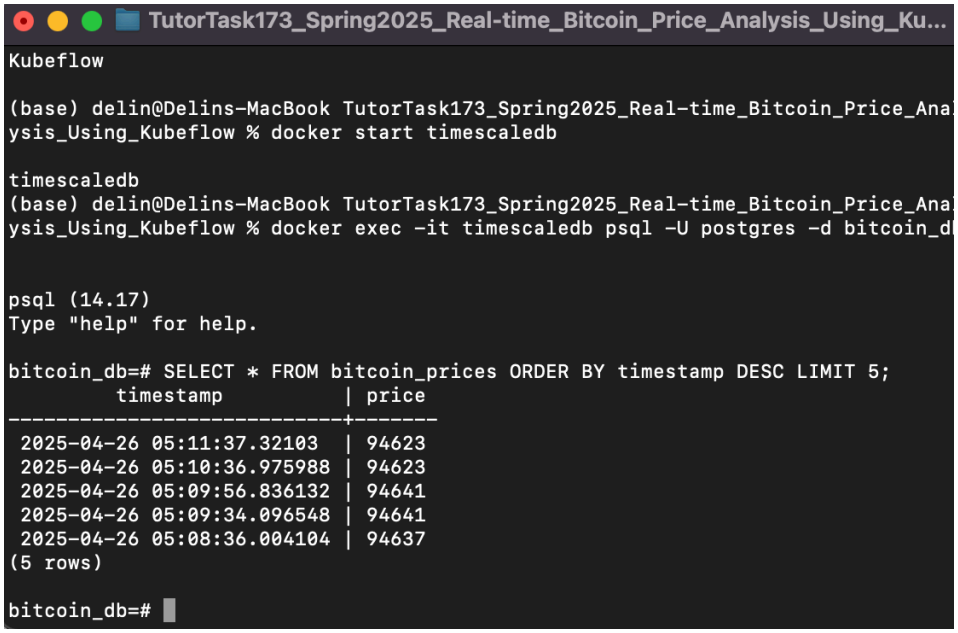
Description: Queried the database to confirm that new Bitcoin prices are inserted correctly.

SQL Query Used:

```
SELECT * FROM bitcoin_prices ORDER BY timestamp DESC LIMIT 5;
```

Result:

- Confirmed that Bitcoin prices were fetched and inserted every minute.



9. Progress so far

Steps	Description
Set up Minikube	Initialized a Kubernetes cluster locally using Docker as the driver, making it ready to host Kubeflow services.
Deployed Kubeflow	Successfully deployed Kubeflow components (pipelines, metadata server, UI, etc.) into the Minikube Kubernetes cluster.
Built Docker Image for Fetcher	Created a Docker image (bitcoin-fetcher:v1) for the Python script that fetches real-time Bitcoin prices via API.
Wrote and Compiled Kubeflow Pipeline	Developed a Python-based Kubeflow pipeline using kfp.dsl, compiled it into a deployable YAML (bitcoin_pipeline.yaml).
Uploaded Pipeline YAML to UI	Uploaded the compiled YAML to the Kubeflow Pipelines UI to register the Bitcoin price-fetching workflow.

Created a Recurring Run	Configured a recurring run to trigger the Bitcoin price fetch every 1 minute automatically through the Kubeflow UI.
Database Connection (Docker / Kubernetes Pod)	Connected to the TimescaleDB/MySQL database pod to verify that Bitcoin prices were correctly inserted.
Verified Price Insertion	Ran SQL queries to confirm that new Bitcoin price records were being stored successfully in the database table.

10. Next Steps

Next Step	Description
1. Exploratory Data Analysis (EDA)	Analyze Bitcoin price trends using Pandas, Matplotlib, Seaborn.
2. Train Basic Predictive Model	ML model or price forecasting.
3. Deploy Model with KFServing	Deploy the model as a live API for real-time predictions.
4. Visualization Report	Create plots and visualizations showing Bitcoin price trend analysis.
5. Final Report + Presentation	Compile screenshots + code walkthrough into final deliverable.